

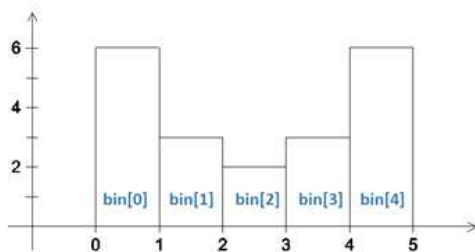
2026.1 Multicore Computing, Project #3

(Due : 11:59pm, May 26)

Submission Rule

1. Create a directory {studentID#}_proj3 (example: 20203601_proj2).
2. Insert into the directory:
 - (i) 'C with OpenMP' source code proj3.c that generates [count for each bin] and [execution time for each algorithm (0,1-1,1-2,2-1,2-2,3-1,3-2) using {2,4,6,8,10,12} threads
 - (ii) a document that contains (a) entire source code, (b) program output, (c) core OpenMP code part (including for-loop block) for each algorithm {0,1-1,1-2,2-1,2-2,3-1,3-2} with brief explanation on how you implemented for each algorithm, and (d) you interpretation and analysis of the results.
 - (iii) insert a demo video file (.mp4 format) where you show (a) compilation, (b) execution, and (c) explanation on how you implemented for each algorithm (show the OpenMP code part for the algorithm you are explaining in the video)
3. zip the directory {studentID#}_proj3 and submit the zip file into eClass homework board.
* If possible, use quad-core/hexa-core/octa-core CPU (or CPU with more cores) rather than dual-core CPU for your experimentation, which will better show the performance gains of the parallelism.

• 1.3, 2.9, 0.4, 0.3, 1.3, 4.4, 1.7, 0.4, 3.2, 0.3, 4.9, 2.4,
3.1, 4.4, 3.9, 0.4, 4.2, 4.5, 4.9, 0.9



[Problem 1] Write a C/C++ code using OpenMP for Serial and Parallel Histogram Construction.

* Problem:

- generate (1024x1024x1024) random floating point numbers between 0 and 10.
- create 10 bins: 0-1, 1-2, ..., 9-10, and construct a histogram as shown in above figure.
- Output: count for each bin and execution time/speedup for each algorithm {0, 1-1, 1-2, 2-1, 2-2, 3-1, 3-2}

* Output example

[0-1): 112730825

[1-2): 91084372

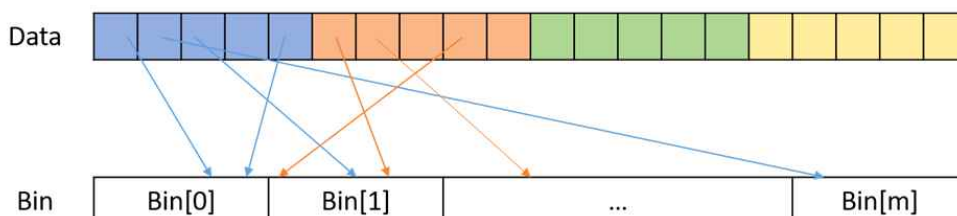
...

[9-10]: 106240811

Algorithm (2 threads)	0	1-1	1-2	2-1	2-2	3-1	3-2
Execution time	ms	ms	ms	ms	ms	ms	ms
speedup	1.0x	x	x	x	x	x	x

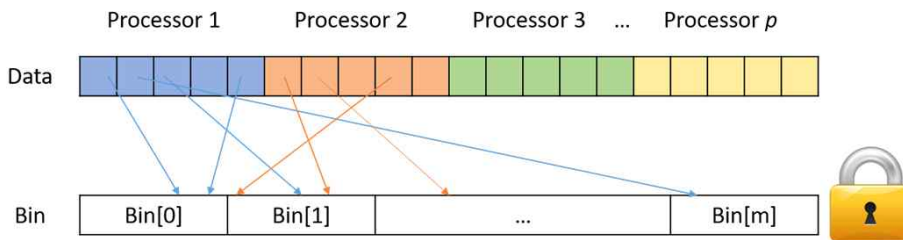
Print this table for {2,4,6,8,10,12} threads

Algorithm 0. Serial Algorithm using a single thread

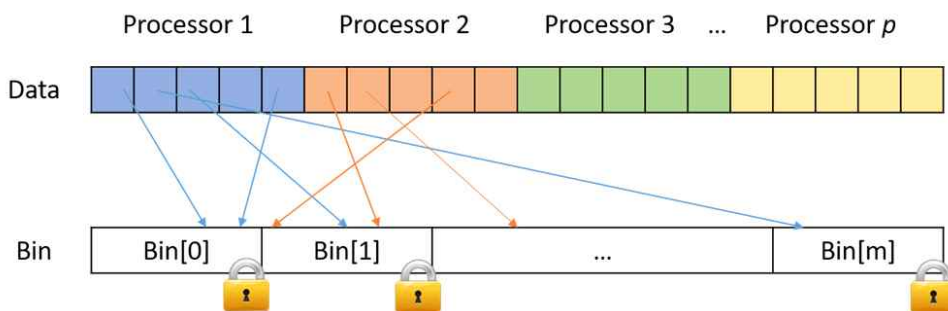


Algorithm 1. Parallel Algorithm with critical section or locks

1-1. Parallel Algorithm with critical section for accessing the bins

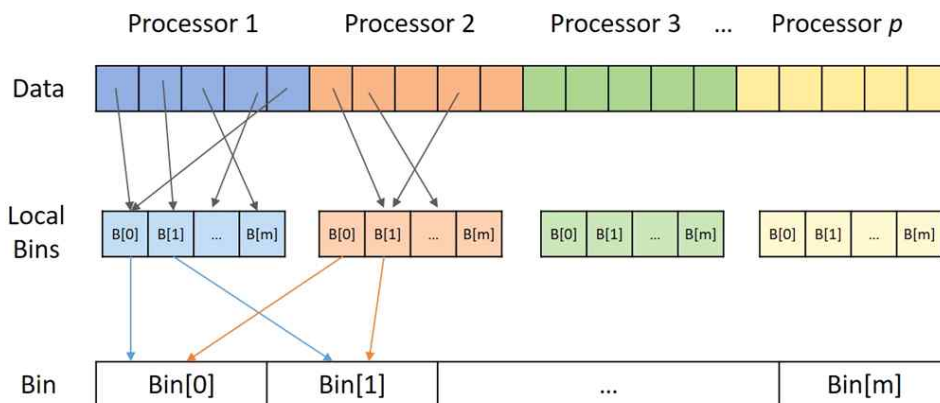


1-2. Parallel Algorithm with locking for each bin

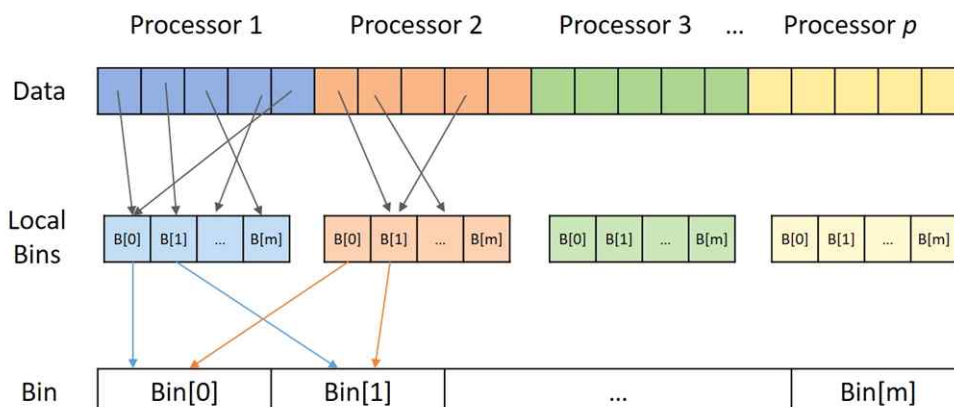


Algorithm 2. Parallel Algorithm with local bins

2-1. Parallel Algorithm with local bins (collecting the local bins in serial)

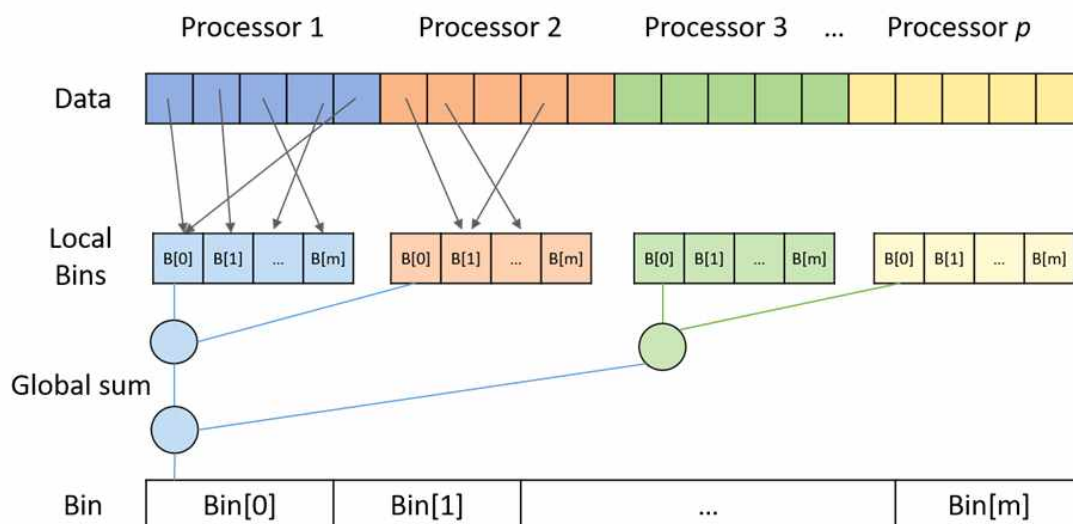


2-2. Parallel Algorithm with local bins (collecting the local bins in parallel)



Algorithm 3. Parallel Algorithm with the reduction

3-1. Parallel Algorithm with the reduction (doing reduction for each bin one by one)



3-2. Parallel Algorithm with the reduction (doing reduction for multiple bins in parallel)

